COMPUTER ENGINEERING

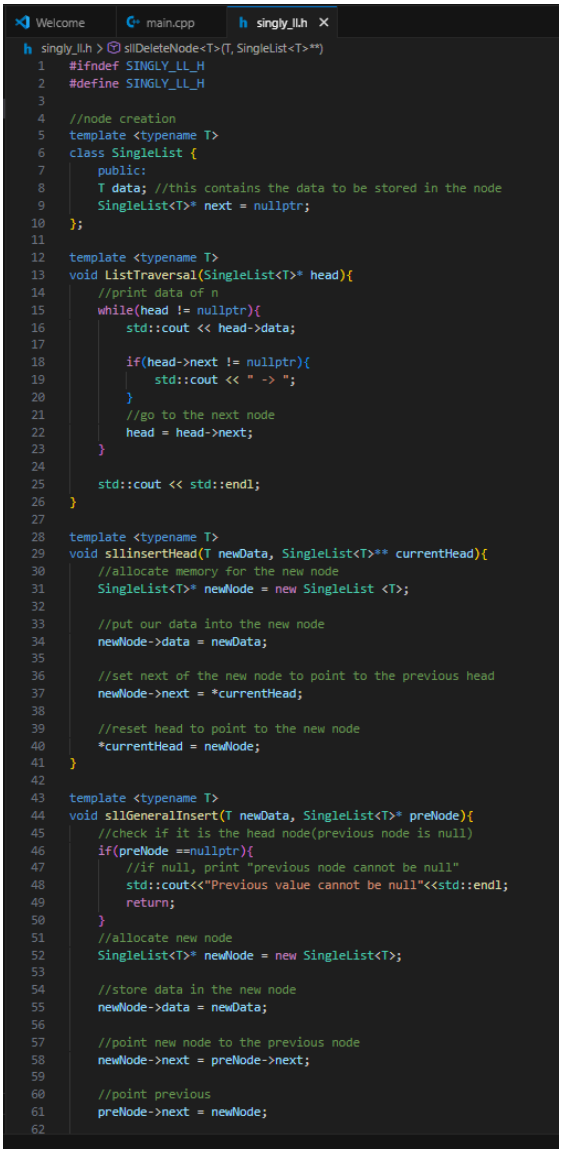
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	07/22/26	**DATE SUBMITTED:**	07/22/26
NAME:	Villaruel, Prince Benjamin S<	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 Creating a Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?



```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

//node creation
template <typename T>
class SingleList {
    public:
    T data; //this contains the data to be stored in the node
    SingleList<T>* next = nullptr;
};

template <typename T>
void ListTraversal(SingleList<T>* head){
    //print data of n
    while(head != nullptr){
        std::cout << head->data;

        if(head->next != nullptr){
            std::cout << " -> ";
        }
        //go to the next node
        head = head->next;
    }

    std::cout << std::endl;
}

template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead){
    //allocate memory for the new node
    SingleList<T>* newNode = new SingleList <T>;

    //put our data into the new node
    newNode->data = newData;

    //set next of the new node to point to the previous head
    newNode->next = *currentHead;

    //reset head to point to the new node
    *currentHead = newNode;
}

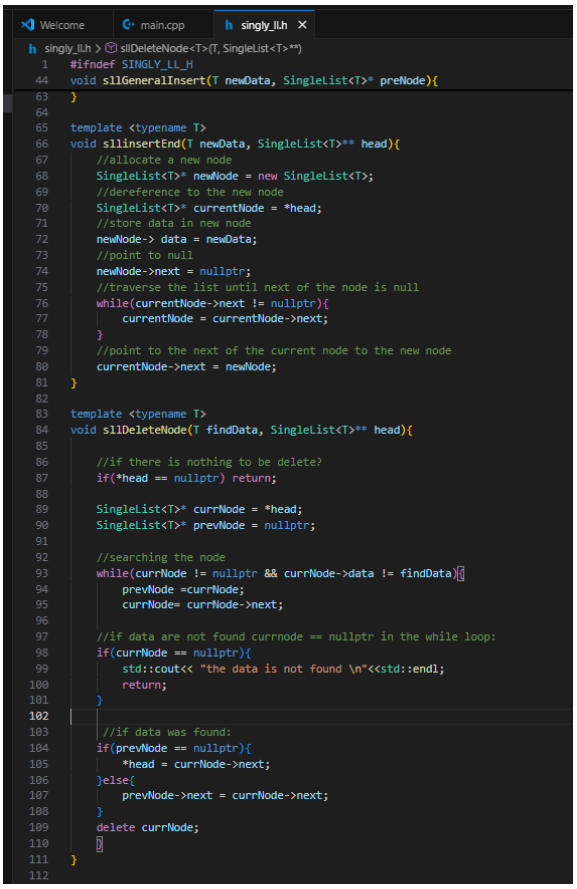
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* preNode){
    //check if it is the head node(previous node is null)
    if(preNode ==nullptr){
        //if null, print "previous node cannot be null"
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //allocate new node
    SingleList<T>* newNode = new SingleList<T>;

    //store data in the new node
    newNode->data = newData;

    //point new node to the previous node
    newNode->next = preNode->next;

    //point previous
    preNode->next = newNode;

```



```
#ifndef SINGLY_LL_H
void sllGeneralInsert(T newData, SingleList<T>* preNode){
}

template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    //allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //dereference to the new node
    SingleList<T>* currentNode = *head;
    //store data in new node
    newNode-> data = newData;
    //point to null
    newNode->next = nullptr;
    //traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }
    //point to the next of the current node to the new node
    currentNode->next = newNode;
}

template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){

    //if there is nothing to be delete?
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //searching the node
    while(currNode != nullptr && currNode->data != findData){
        prevNode =currNode;
        currNode= currNode->next;

    //if data are not found currnode == nullptr in the while loop:
    if(currNode == nullptr){
        std::cout<< "the data is not found \n"<<std::endl;
        return;
    }

    //if data was found:
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    delete currNode;
    }
}

```



```
template <typename T>
void sllDeleteList(SingleList<T> **head){
    SingleList<T>* current = *head;

    while(current != nullptr){
        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}


#endif
```

```cpp
main.cpp > main()
4   int main(){
18
19      //storing a value inside the node
20      head->data = 'P';
21      second->data = 'B';
22      third->data = 'S';
23      fourth->data = 'V';
24
25      //linking the nodes
26      head->next = second;
27      second->next = third;
28      third->next = fourth;
29      fourth->next = nullptr;
30
31      std::cout<< "Testing of traversal: ";
32      ListTraversal(head);
33
34      std::cout<<"Testing of sllinsertHead: \n";
35      sllinsertHead('Q', &head);
36      ListTraversal(head);
37
38      std::cout<<"Testing of sllGeneralInsert:\n";
39      sllGeneralInsert('E', head->next->next);
40      ListTraversal(head);
41
42      std::cout<<"Testing of sllinsertEnd:\n";
43      sllinsertEnd('C', &head);
44      ListTraversal(head);
45
46      std::cout<<"Testing of deleting the node:\n";
47      sllDeleteNode('C', &head);
48      ListTraversal(head);
49      sllDeleteNode('E', &head);
50      ListTraversal(head);
51
52      std::cout<<"deleting ALL nodes:\n";
53      sllDeleteList(&head);
54      ListTraversal(head);
55
56      return 0;
57
58  }
59
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Linked Li
Testing of traversal: P -> B -> S -> V
Testing of sllinsertHead:
Q -> P -> B -> S -> V
Testing of sllGeneralInsert:
Q -> P -> B -> E -> S -> V
Testing of sllinsertEnd:
Q -> P -> B -> E -> S -> V -> C
Testing of deleting the node:
Q -> B -> S -> C
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Linked Li
```

Reflection

In this singly linked list activity, I learned a lot about paying close attention to small details in code. I noticed that simple mistakes, like a missing semicolon or a misplaced curly brace, can cause big errors, even if the program still runs without showing an error. This made me realize that not all bugs are caught by the compiler, some just quietly cause the program to behave the wrong way. I also got a better understanding of how pointers work, especially when inserting or deleting nodes, since one wrong step can break the connection between nodes in the list, and I learned why functions modify the head, like insertion or deletion, need a pointer to a pointer instead of just the head itself. Working through the delete functions also reminded me why memory management matters, since every node created with new needs to be properly removed with delete, or it just stays taking up memory. Overall, this activity helped me become more careful when checking my code, a lesson I can for sure carry over into future laboratories.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.